

WEEK-7:

Experiment 1: Smart Counter with Theme Toggle

Aim: To understand **useState, events, and conditional rendering** by building a counter with theme switching.

Description:

This experiment demonstrates how React updates UI dynamically using state. The counter increments/decrements values and switches between light and dark themes.

Procedure:

1. Create a new React app using create-react-app.
2. Create a component named Counter.
3. Use useState to store:
 - count value
 - theme (light/dark)
4. Add buttons:
 - Increment
 - Decrement
 - Reset
 - Toggle Theme
5. Apply conditional styling based on theme.
6. Disable decrement when count is 0.
7. Run the application.

Outcome:

- Students understand **state updates**
- Learn **event handling**
- UI changes dynamically based on state

Code:

1.Counter.js:

```
import React, { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  const [theme, setTheme] = useState("light");
  const styles = {
    textAlign: "center",
    padding: "20px",
    backgroundColor: theme === "light" ? "#fff" : "#333",
    color: theme === "light" ? "#000" : "#fff",
    minHeight: "100vh"
  };
}
```

Laboratory Record
of Enterprise Application
Development Lab

Roll No.: 160124737077
Exp No.: _____
Sheet No.: _____
Date: 18/03/2026

```
return (  
  <div style={styles}>  
    <h1>Smart Counter</h1>  
    <h2>{count}</h2>  
    <button onClick={() => setCount(count + 1)}>Increment</button>  
    <button  
      onClick={() => setCount(count - 1)}  
      disabled={count === 0}    >  
      Decrement  
    </button>  
    <button onClick={() => setCount(0)}>Reset</button>    <br /><br />  
    <button onClick={() =>  
      setTheme(theme === "light" ? "dark" : "light")  
    }>  
      Toggle Theme  
    </button>  
  </div>  
}
```

export default Counter;

2.App.js:

```
import React from "react";  
import Counter from "./Counter";
```

```
function App() {  
  { return <Counter />;  
}
```

export default App;

Output:



Experiment 2: Timer / Stopwatch with Lap

Aim: To implement a timer using **useEffect** and **setInterval**.

Description:

This experiment simulates a real-world stopwatch with start, pause, reset, and lap features.

Procedure:

1. Create a component Timer.
2. Use **useState**:
 - seconds
 - isRunning
 - laps array
3. Use **useEffect** to:
 - Start interval when running
 - Stop interval when paused
4. Add buttons:
 - Start
 - Pause
 - Reset
 - Lap
5. Convert seconds into **MM:SS** format.
6. Display lap times in a list.
7. Run and test functionality.

Outcome:

- Understand **side effects (useEffect)**
- Learn **cleanup functions**
- Handle real-time updates

Code:

1. Timer.js:

```
import React, { useState, useEffect } from "react";

function Timer() {

  const [seconds, setSeconds] = useState(0);

  const [isRunning, setIsRunning] = useState(false); const [laps, setLaps] = useState([]);

  // useEffect for timer  useEffect(() => { let interval = null; if (isRunning)

    { interval = setInterval(() => { setSeconds(prev => prev + 1);
```

Laboratory Record
of Enterprise Application
Development Lab

Roll No.: 160124737077
Exp No.: _____
Sheet No.: _____
Date: 18/03/2026

```
    }, 1000); } else {    clearInterval(interval); }

return () => clearInterval(interval); }, [isRunning]); // Format MM:SS const formatTime = (sec) =>

{ const minutes = Math.floor(sec / 60); const seconds = sec % 60; return

( String(minutes).padStart(2, "0") + ":" + String(seconds).padStart(2, "0") ); }; return (

<div style={{ textAlign: "center", padding: "20px" }}>    <h1>Stopwatch</h1>

    <h2>{formatTime(seconds)}</h2>

    <button onClick={() => setIsRunning(true)}>Start</button>

    <button onClick={() => setIsRunning(false)}>Pause</button>    <button onClick={() =>

        { setSeconds(0);    setLaps([]);    setIsRunning(false);    }}>    Reset    </button>

    <button onClick={() => setLaps([...laps, seconds])}>    Lap    </button>    <h3>Laps:</h3>

    <ul>        {laps.map((lap, index) => (        <li key={index}>{formatTime(lap)}</li>        ))}    </ul>

</div> );}

export default Timer;
```

2. App.js:

```
import React from "react";

import Timer from "./Timer";

function App() {

    return <Timer />;

}

export default App;
```

Output:

